Nhận dạng chữ số viết tay MNIST

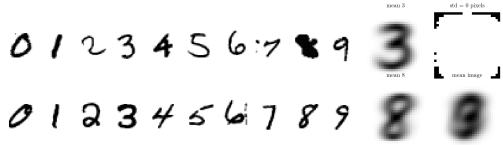
Bốn bộ phân lớp tự cài đặt, một mạng tích chập nhỏ,
đo trên cùng một giao thức và đối chiếu với scikit-learn

Trần Trinh Nhân (24280088) Lâm Nhật Tiến (24280029)

Nhận dạng mẫu – Đồ án giữa kỳ

Đọc một chữ số 28×28 và trả lời nó là số nào

- MNIST: 60 000 ảnh huấn luyện, 10 000 ảnh test, 10 lớp; mỗi ảnh có 784 pixel xám.
- Câu hỏi: các họ phương pháp khác nhau cho kết quả ra sao khi cùng làm một bài và cùng đo một cách?
- Mở rộng: Fashion-MNIST, EMNIST Digits, tăng cường dữ liệu, một CNN nhỏ và một trình khám phá quá trình tính toán trong trình duyệt.



Giao thức: không rò rỉ dữ liệu, chạy lại được bằng một lệnh

Ba tập, ba vai

Train để học, validation để chọn, test để báo cáo. Hàm tìm siêu tham số không nhận tập test; một kiểm thử tự động giữ điều đó.

Chuẩn hóa chỉ từ train

Trung bình và độ lệch chuẩn tính trên 50 000 ảnh train, dùng lại nguyên si cho validation, test và trình duyệt.

```
prepare → chia 50 000/10 000
tune → chọn trên validation
train → huấn luyện lại
evaluate → test, một lần
export → ONNX + kiểm tra số
report → bảng, hình
```

Năm mô hình, năm cách nhìn khác nhau về chữ số

Gaussian Naïve Bayes	mỗi pixel là một phân phối chuẩn riêng theo lớp, các pixel độc lập
Hồi quy softmax	một mẫu $\boldsymbol{w}_k$ cho mỗi lớp, học bằng gradient descent trên cross-entropy
MLP	hai lớp ẩn ReLU, lan truyền ngược viết tay, momentum, dừng sớm
AdaBoost-SAMME	hàng trăm cây quyết định có trọng số; cây sau tập trung vào ảnh cây trước làm sai
CNN nhỏ	hai khối tích chập + max-pool + ReLU, rồi lớp dày đặc; im2col và gradient viết tay

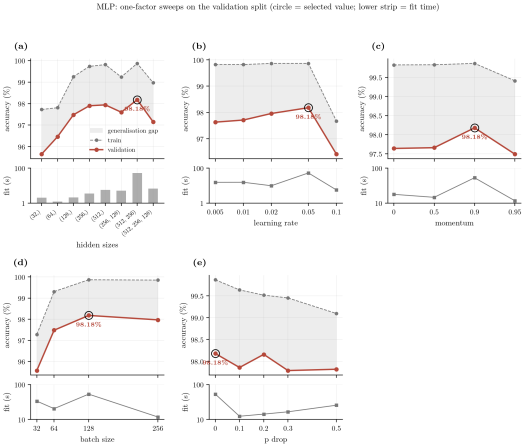
Phần lõi tự viết, thư viện chỉ để kiểm tra

- Ước lượng tham số, loss, gradient và cập nhật viết tay bằng NumPy; MLP và CNN dùng PyTorch *chỉ làm kho tensor*: không autograd, không `nn.Module`, không `optim`, không `F.conv2d`, không `F.max_pool2d`.
- scikit-learn chỉ để đối chiếu.

107 kiểm thử tự động

Gradient giải tích khớp sai phân hữu hạn; backprop và gradient CNN khớp autograd; convolution khớp vòng lặp ngây thơ; cây có trọng số khớp `DecisionTreeClassifier`.

Chọn siêu tham số: quét từng tham số một, chọn bằng validation

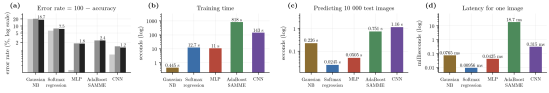


- Công cụ: **tự viết**, không dùng Optuna. Mỗi tham số được quét riêng theo từng bước (coordinate descent), chi phí là tổng số giá trị thay vì tích.
- Chọn theo accuracy trên validation; mọi lần chạy được ghi lại bằng Weights & Biases.
- Đồ thị trái chính là hình "đổi một siêu tham số, giữ nguyên phần còn lại" mà đề bài yêu cầu.
- Giới hạn: không xét tương tác giữa các siêu tham số.

Kết quả trên tập test

Mô hình	Test accuracy
Gaussian NB	81.32 %
Softmax	92.52 %
MLP	98.24 %
AdaBoost-SAMME	97.60 %
CNN nhỏ	98.80 %

Một seed (42), 10 000 ảnh test. Khoảng tin cậy 95% mỗi số rộng cỡ ± 0.3 điểm; so sánh cặp bằng kiểm định McNemar chính xác.

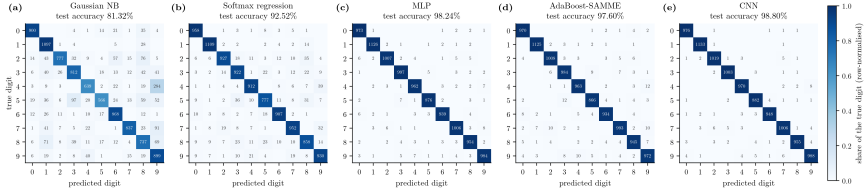


Độ chính xác không đi cùng thời gian

Mô hình	Test (%)	Huấn luyện (s)	Một ảnh (ms)
Gaussian NB	81.32	0.4	0.076
Softmax	92.52	12.7	0.010
MLP	98.24	11.0	0.042
AdaBoost-SAMME	97.60	817.9	18.705
CNN nhỏ	98.80	143.0	0.315

Toàn bộ trên CPU (không dùng GPU): các phép tính dùng BLAS đa luồng của NumPy/PyTorch; GPU không dùng vì bản PyTorch cài là bản CPU. AdaBoost duyệt hàng trăm cây cho mỗi ảnh nên chậm nhất.

Lỗi tập trung ở các cặp có nét giống nhau

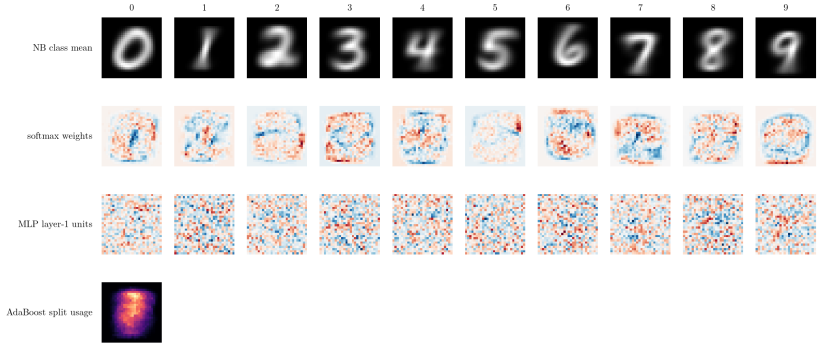


Cùng siêu tham số, kết quả sát scikit-learn

Mô hình	Bản của nhóm	scikit-learn	Chênh
Gaussian NB	81.32	81.32	0.00
Softmax	92.52	91.86	0.66
MLP	98.24	98.02	0.22
AdaBoost-SAMME	97.60	97.50	0.10

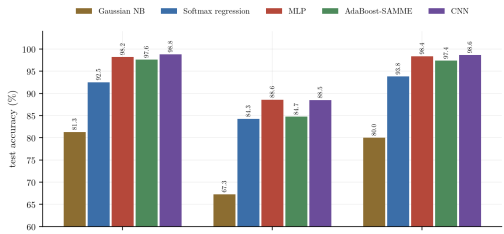
Softmax chênh vì L-BFGS của scikit-learn chạm giới hạn vòng lặp, không dừng sớm như bản của nhóm. Thời gian AdaBoost không so trực tiếp được vì hai bên chia ngưỡng khác nhau.

Mô hình học được gì: có cái đọc được bằng mắt, có cái thì không



Mở rộng 1: Fashion-MNIST và EMNIST Digits

- Cùng mã, cùng giao thức, cùng bộ đánh giá; chia train/val/test riêng cho từng tập và báo cáo tách biệt.
- Tên lớp khác nhau (áo, giày… so với chữ số), nên mô hình huấn luyện trên một tập *không* được đánh giá trên tập khác nếu định nghĩa lớp khác nhau; chương trình từ chối phép đó.
- Chỉ có một phép kiểm tra chéo hợp lệ: MNIST → EMNIST Digits (cùng 10 chữ số), báo cáo riêng cho mô hình huấn luyện trên MNIST: Naïve Bayes 49.80 % (so với 80.03 % khi huấn luyện trên EMNIST), CNN 92.15 % (so với 98.64 %).



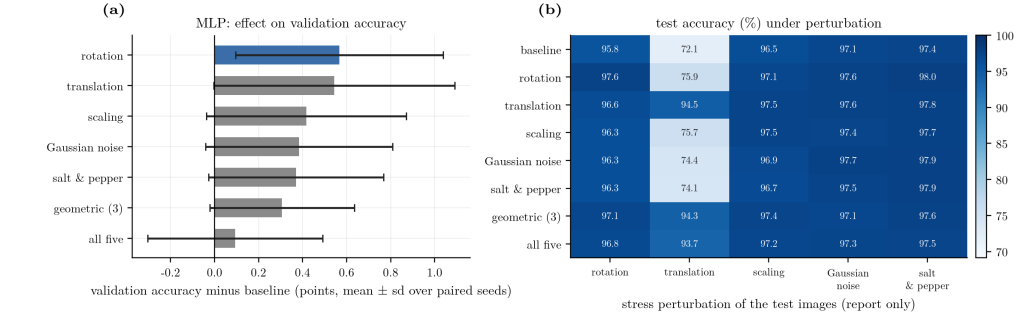
Siêu tham số của Fashion-MNIST và EMNIST Digits chọn bằng quét rút gọn một seed; chênh lệch nhỏ giữa MLP và CNN ở đó chưa đủ để kết luận.

Mở rộng 2: tăng cường dữ liệu, thí nghiệm có đối chứng

- Năm phép: xoay, dịch, co giãn, nhiễu Gauss, nhiễu muối tiêu; chỉ tác động lên tập train.
- Điều kiện: gốc, từng phép riêng lẻ, nhóm hình học, cả năm; nhiều seed ghép cặp.
- Chọn điều kiện *chỉ* theo validation; test dùng để báo cáo và đo độ bền với nhiễu mạnh hơn lúc huấn luyện.

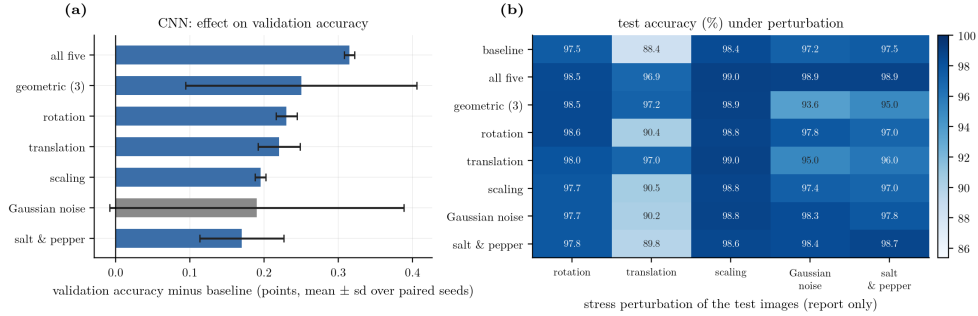


Tăng cường dữ liệu: phép nào giúp, phép nào hại



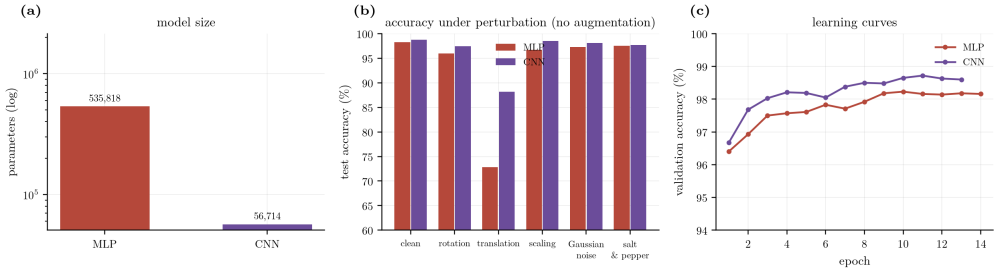
MLP, 3 seed. Mọi phép cho validation cao hơn gốc nhưng cùng bậc với độ lệch chuẩn của hiệu ghép cặp; tác dụng rõ nhất là độ bền với *đúng loại* biến đổi (ví dụ dịch). "Cả năm phép" không hơn các phép riêng lẻ.

Tăng cường dữ liệu cho CNN: có phép làm giảm độ bền



CNN, 2 seed (độ lệch chuẩn kém tin cậy). Cả năm phép được chọn theo validation. Chỉ dịch hoặc nhóm hình học làm CNN kém bền hơn với nhiễu Gauss và muối tiêu. Mỗi điều kiện có dữ liệu gấp đôi nên loại biến đổi và lượng dữ liệu bị trộn lẫn.

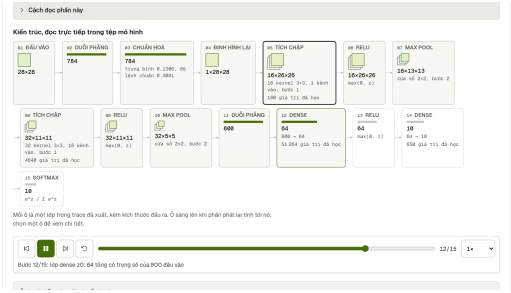
Mở rộng 3: CNN nhỏ so với MLP



CNN: 98.80 % so với MLP 98.24 %. Kích thước, kênh, kernel và lớp dày đặc đều lấy từ `CNN.describe()` của mô hình thật; mọi tham số đã quét được ghi lại.

Trình khám phá quá trình tính toán chạy trong trình duyệt

- Python sinh một *hợp đồng vết* (`writerhand.trace/1`) từ chính mô hình đã huấn luyện: đồ thị lớp, hình dạng tensor, tiền xử lý, tham số, mã băm.
- Trình duyệt chạy ONNX Runtime Web trên ảnh vừa vẽ và vẽ sơ đồ *từ metadata*, không có kiến trúc nào được gõ cứng; đổi mô hình hay ảnh thì kết quả cũ bị loại.
- Mỗi phép tính trung gian được tái dựng từ tham số và đối chiếu với giá trị ONNX.



Trực quan hóa tính toán không phải là giải thích quyết định

Phân biệt

Bản đồ kích hoạt của một lớp cho thấy lớp đó *tính ra gì* với ảnh này; nó không chứng minh pixel hay bộ lọc nào *gây ra* quyết định. Với softmax và Naïve Bayes, phân rã $x_i W_{ik}$ hay log-likelihood theo pixel là chính xác (mô hình tuyến tính/cộng tính), và được ghi nhãn như vậy.

AdaBoost

Các con số cộng dồn là *tỉ lệ phiếu có trọng số*, không phải xác suất đã hiệu chỉnh.

Kết luận

Điều rút ra

- CNN cho kết quả tốt nhất trên MNIST và EMNIST Digits (khoảng một phần mười số tham số của MLP); trên Fashion-MNIST CNN và MLP ngang nhau.
- AdaBoost sát nhưng đắt gấp hàng chục lần khi suy luận.
- Chọn cấu hình có kỷ luật quan trọng ngang chọn thuật toán.

Giới hạn

- Một seed cho các mô hình chính.
- Quét tuần tự chưa xét tương tác siêu tham số.
- Thời gian đo trên một máy CPU.
- Kết quả chỉ nói về các tập đã đo, không phải chữ viết tay nói chung.

Đóng góp, công bố sử dụng AI, liên kết

- Đóng góp 50 %/50 % (Trần Trinh Nhân, Lâm Nhật Tiến), do hai thành viên thống nhất.
- Phần hoàn thiện sau commit `f9a602b` được viết với sự hỗ trợ của Claude Code; số liệu trong báo cáo, bảng và slide đều sinh từ tệp kết quả do chương trình ghi ra.
- Mã nguồn: `https://github.com/TranTrinhNhan1/WriteRHand`
- Demo: `https://trantrinhnhan1.github.io/WriteRHand/`
- Mô hình: `https://huggingface.co/TranTrinhNhan1111/WriteRHand-MNIST`

Cảm ơn thầy cô và các bạn đã lắng nghe.